

Distributed Web Crawling and Parallel PageRank Computation

Milestone 2 Report

Parallel PageRank Computation

Course: Parallel and Distributed Computing

Group Members

Abdullah Irfan	29266
Ameer Hamza	28308
Muhammad Khizer	28991
Rameez Hoda	29057

Note on AI Assistance: *This report and the corresponding code were refined, enhanced, and made grammatically correct using Claude Sonnet 4.5 (Anthropic).*

April 12, 2026

Contents

1	Project Overview	3
1.1	What the Project is About	3
1.2	Recap of Milestone 1	3
1.3	What Milestone 2 is About	4
1.4	Note on Graph Size Limitations	4
2	System Architecture	5
2.1	Pipeline Overview	5
2.2	File Structure	5
2.3	Data Flow	5
2.4	Technology Stack	6
3	Detailed Module Descriptions	7
3.1	<code>pagerank_config.py</code> — Central Configuration	7
3.1.1	Purpose and Design	7
3.1.2	Implementation	7
3.1.3	Design Decisions	7
3.2	<code>graph_loader.py</code> — Graph Loading and Preparation	7
3.2.1	Purpose and Design	7
3.2.2	Implementation	7
3.2.3	Design Decisions	8
3.3	<code>pagerank_sequential.py</code> — Sequential Baselines	8
3.3.1	Purpose and Design	8
3.3.2	Implementation	8
3.3.3	Design Decisions	9
3.4	<code>pagerank_parallel.py</code> — Parallel PageRank Core	9
3.4.1	Purpose and Design	9
3.4.2	Implementation	9
3.4.3	Design Decisions	10
3.5	<code>performance_analysis.py</code> — Metrics and Reporting	10
3.5.1	Purpose and Design	10
3.5.2	Implementation	10
3.5.3	Design Decisions	11
3.6	<code>main_pagerank.py</code> — Pipeline Orchestrator	11
3.6.1	Purpose and Design	11
3.6.2	Implementation	11
3.6.3	Design Decisions	11
4	Experimental Results	12
4.1	Test Configuration Overview	12
4.2	Baseline Results (Test 1)	12
4.3	Worker Scaling Analysis (Tests 2, 3, 4)	13
4.4	Strategy Comparison (Tests 5, 6)	14
4.5	Termination Strategy Comparison (Tests 7, 8, 9)	15
4.6	Graph Size Comparison (Tests 10, 11)	16
4.7	Damping Factor Sensitivity (Tests 12, 13, 14)	17
4.8	Communication Overhead Analysis	19

4.9	Load Imbalance Analysis	19
4.10	Convergence Rate Analysis	20
4.11	Amdahl's Law Analysis	20
5	Work Done So Far and Remaining Tasks	22
5.1	Milestone 1 — Completed	22
5.2	Milestone 2 — Completed	22
5.3	Milestone 3 — Remaining Tasks	22
6	Conclusion	24

1 Project Overview

1.1 What the Project is About

This project is about building a system that can crawl a large number of web pages, build a graph from those pages, and then run the PageRank algorithm on that graph in parallel. The main idea is not just to get it working, but to do it in a way that uses multiple CPU cores or even multiple machines at the same time, which is the core of Parallel and Distributed Computing.

The web can be thought of as a directed graph where each page is a node and every hyperlink from one page to another is a directed edge. PageRank is an algorithm originally developed by Google that assigns an importance score to every node based on how many other important nodes link to it. Computing PageRank on a large graph with millions of nodes takes a long time if done sequentially, which is why parallelism matters here.

The project is divided into three milestones that build on each other:

- **Milestone 1** builds the web graph by crawling pages and extracting links in parallel.
- **Milestone 2** takes that graph and runs the PageRank algorithm in parallel across multiple workers.
- **Milestone 3** extends the system to handle new pages being added after PageRank has already been computed, and does a full performance evaluation.

The system is implemented using **Ray**, a Python-based parallel runtime that makes it easy to run functions on multiple CPU cores and manage shared state between them.

1.2 Recap of Milestone 1

In Milestone 1, we implemented a parallel web crawling pipeline that:

- Used **Ray workers** to concurrently fetch and parse web pages from Wikipedia, demonstrating task parallelism.
- Extracted hyperlinks to construct a **directed graph** where nodes represent pages and edges represent hyperlinks.
- Employed **shared-state coordination** via a centralised URL frontier to filter duplicate URLs and manage crawl scope.
- Applied **depth and domain constraints** to produce deterministic, reproducible graph workloads.
- Serialised the resulting NetworkX DiGraph as a pickle file (`web_graph.pkl`) for downstream consumption by the PageRank module.

Two graph configurations were produced:

- **Config 1 (Small)**: 199 seed pages $\rightarrow$ 15,657 nodes, 25,576 edges
- **Config 3 (Medium)**: 496 seed pages $\rightarrow$ 37,074 nodes, 67,667 edges

Both graphs exhibit a high percentage of dangling nodes ($\approx 98.7\%$), which is characteristic of partial crawls where many discovered URLs were not themselves crawled due to depth or domain limits.

1.3 What Milestone 2 is About

Milestone 2 is the **core computational component** of the project. It takes the directed web graph produced by Milestone 1 and applies the PageRank algorithm to compute importance scores for every node. The key objectives are:

1. **Apply data parallelism** to an iterative graph algorithm using Ray.
2. **Compare execution strategies:** centralised aggregation versus distributed reduction.
3. **Compare termination strategies:** convergence-based versus fixed iteration count.
4. **Study scalability** by varying the number of workers (1, 2, 4).
5. **Analyse sensitivity** to algorithm parameters (damping factor) and graph size.
6. **Measure and report** comprehensive performance metrics: speedup, efficiency, communication overhead, load imbalance, convergence rate, and memory usage.

PDC focus: data partitioning, synchronisation barriers, iterative parallel algorithms, Amdahl’s Law analysis.

1.4 Note on Graph Size Limitations

During our experimentation, we intended to produce a significantly larger graph (100,000+ nodes) to better demonstrate parallel speedup. However, the **CommonCrawl CDX server**, which we used as an external data source for discovering seed URLs, experienced prolonged downtime and rate-limiting issues during our testing period. As a result, we were unable to generate graphs beyond Config 3 (37,074 nodes). While the current graph sizes are sufficient to demonstrate all PDC concepts and produce meaningful analysis, the observed speedup values of less than $1.0\times$ are a direct consequence of this size limitation. This is itself an important PDC finding: parallelisation overhead dominates when workloads are too small relative to the framework’s fixed costs. A larger graph would shift the computation-to-overhead ratio in favour of parallel execution.

2 System Architecture

2.1 Pipeline Overview

The Milestone 2 system follows a six-phase pipeline, orchestrated by `main_pagerank.py`:

ALL PHASES
Phase 1: Load graph from Milestone 1 pickle file
Phase 2a: Custom sequential PageRank (baseline for speedup)
Phase 2b: <u>NetworkX</u> PageRank (correctness reference)
Phase 3: Parallel PageRank — <u>Centralised</u> Aggregation
Phase 4: Parallel PageRank — Distributed Reduction
Phase 5: Validate correctness (parallel $\approx$ sequential)
Phase 6: Full performance report with all metrics

Figure 1: Six-phase pipeline for the Milestone 2 PageRank system.

Each phase depends on the preceding one. The sequential baselines (Phases 2a, 2b) establish ground truth timings and correctness references against which the parallel strategies (Phases 3, 4) are evaluated. The validation phase ensures that parallel computation does not introduce numerical divergence beyond an acceptable tolerance (10^{-4}).

2.2 File Structure

```

1 project/
2 |-- (Milestone 1 .py files)
3 |-- output_config1/
4 |   +-- web_graph.pkl           # Small graph (15,657 nodes)
5 |-- output_config3/
6 |   +-- web_graph.pkl           # Medium graph (37,074 nodes)
7 |-- pagerank_config.py          # Central configuration dataclass
8 |-- graph_loader.py             # Graph loading and preparation
9 |-- pagerank_sequential.py      # Sequential baselines
10 |-- pagerank_parallel.py        # Ray parallel strategies
11 |-- performance_analysis.py     # Metrics, reporting, Amdahl analysis
12 |-- main_pagerank.py           # CLI entry point, pipeline orchestrator
13 |-- requirements_m2.txt         # Python dependencies

```

Milestone 2 File Structure

2.3 Data Flow

1. `graph_loader.py` reads the NetworkX DiGraph from the pickle file and converts it into integer-indexed adjacency structures (in-link lists, out-degree arrays, dangling node sets) for efficient computation and reduced inter-process communication.
2. `pagerank_sequential.py` computes the baseline scores using both a custom iterative implementation and NetworkX's built-in SciPy-based solver.

3. `pagerank_parallel.py` partitions the node set across Ray workers and executes the PageRank iterations in parallel using one of two strategies.
4. `performance_analysis.py` collects all metrics and produces formatted comparison tables, Amdahl's Law projections, and JSON output files.

2.4 Technology Stack

Table 1: Technology stack for Milestone 2

Component	Technology
Language	Python 3.12
Parallel runtime	Ray $\geq$ 2.9.0
Graph library	NetworkX $\geq$ 3.2.0
Numerical backend	NumPy $\geq$ 1.26.0
Memory profiling	psutil
Sparse linear algebra	SciPy (via NetworkX)

3 Detailed Module Descriptions

3.1 pagerank_config.py — Central Configuration

3.1.1 Purpose and Design

This module defines a single `PageRankConfig` dataclass that centralises every tunable parameter. By following the same configuration pattern established in Milestone 1, no magic numbers are hardcoded in algorithm files, and every experiment is fully reproducible from its configuration alone.

3.1.2 Implementation

The dataclass contains five groups of parameters:

- **Graph input:** path to the pickle file produced by Milestone 1.
- **Algorithm parameters:** damping factor ($d = 0.85$ default), maximum iterations (100), convergence threshold (10^{-6}), and whether to use convergence-based termination.
- **Parallelism settings:** number of Ray workers and number of graph partitions (set equal by default for balanced load).
- **Execution strategy:** "centralized", "distributed", or "both".
- **Output settings:** output directory and whether to save final scores.

3.1.3 Design Decisions

Using a dataclass rather than a dictionary provides type safety and IDE auto-completion. Setting `num_partitions = num_workers` by default ensures one partition per worker, which is the optimal mapping for range-based partitioning with roughly equal node counts per chunk.

3.2 graph_loader.py — Graph Loading and Preparation

3.2.1 Purpose and Design

This module bridges Milestone 1 and Milestone 2. It loads the NetworkX `DiGraph` from the pickle file produced by the crawler and converts it into the integer-indexed data structures that the PageRank workers need.

3.2.2 Implementation

The module performs three key transformations:

1. **URL-to-integer mapping:** Every URL string is mapped to an integer index once at load time. All subsequent computation and communication use integer arrays instead of string dictionaries, significantly reducing serialisation overhead in Ray.

2. **In-link index construction:** For each node u , we build a list of all nodes v such that there exists an edge $v \rightarrow u$. This is the transpose adjacency structure that PageRank requires.
3. **Dangling node identification:** Nodes with zero outgoing edges are collected into a set. These nodes absorb rank without redistributing it and require special handling in the PageRank formula.

The `partition_nodes()` function splits the node ID range $[0, N)$ into k roughly equal chunks using simple range-based partitioning. Each chunk is assigned to one Ray worker.

3.2.3 Design Decisions

Integer indexing was chosen over string-based communication because Ray serialises data using Apache Arrow / pickle, and transmitting 37,074 URL strings per iteration would add substantial overhead. With integer indices, the rank vector is a compact list of floats that serialises efficiently.

Range-based partitioning (contiguous node ID ranges) was chosen over hash-based or graph-aware partitioning because it is simple, deterministic, and produces perfectly balanced partition sizes. While graph-aware partitioning could reduce cross-partition edges and thus communication, the additional complexity was not warranted for this graph size.

3.3 pagerank_sequential.py — Sequential Baselines

3.3.1 Purpose and Design

This module provides two sequential PageRank implementations:

1. `networkx_pagerank()`: Uses NetworkX's built-in SciPy-based solver as the **gold-standard correctness reference**.
2. `custom_pagerank()`: Our own iterative implementation using the same algorithm that is parallelised in the next module. This serves as the **timing baseline** for speedup calculations.

3.3.2 Implementation

The custom implementation follows the standard iterative PageRank formula. For each node u at iteration t :

$$r_u^{(t+1)} = \frac{1-d}{N} + d \cdot \frac{S_{\text{dangling}}}{N} + d \sum_{v \in \text{In}(u)} \frac{r_v^{(t)}}{\text{out}(v)} \quad (1)$$

where d is the damping factor, N is the total number of nodes, $S_{\text{dangling}} = \sum_{j \in D} r_j^{(t)}$ is the sum of ranks of all dangling nodes, and $\text{In}(u)$ is the set of nodes with edges pointing to u .

The convergence criterion is the L1 norm of the rank change vector: $\delta^{(t)} = \sum_u |r_u^{(t+1)} - r_u^{(t)}|$. Iteration stops when $\delta < \epsilon$ (default 10^{-6}).

The `validate_scores()` function compares two score dictionaries node-by-node and reports the maximum absolute difference, confirming that the parallel implementation produces numerically equivalent results.

3.3.3 Design Decisions

Maintaining a separate custom sequential implementation (rather than only using NetworkX) is essential because it applies the *exact same* algorithm as the parallel version. Any difference in results between the sequential and parallel runs can therefore be attributed to parallelisation artifacts (floating-point non-associativity, race conditions) rather than algorithmic differences. The per-iteration delta list is recorded for convergence rate analysis in the report.

3.4 pagerank_parallel.py — Parallel PageRank Core

3.4.1 Purpose and Design

This is the **core PDC module**. It implements two parallel execution strategies for the PageRank algorithm using Ray:

1. **Strategy A — Centralised Aggregation:** Workers compute partial rank updates for their assigned node partitions. The driver collects all partial results via `ray.get()` (synchronisation barrier), merges them into a complete rank vector, and broadcasts the updated vector for the next iteration.
2. **Strategy B — Distributed Reduction:** Workers compute partial rank updates and write them directly to a shared `RankActor` (a Ray Actor). The actor accumulates updates from all workers and finalises the iteration when all writes are complete.

3.4.2 Implementation

Shared Ray Task: The `compute_partial_ranks()` function is a `@ray.remote` task that computes new PageRank scores for one partition of nodes. It applies the same formula as the sequential version but only over a subset of nodes. It also records per-task execution time and memory usage for load imbalance analysis.

Strategy A (Centralised):

1. Large read-only structures (in-links, out-degree) are placed in Ray's object store once via `ray.put()`.
2. Each iteration: the driver computes the dangling sum, dispatches k tasks concurrently, waits for all via `ray.get()` (barrier), merges partial results, and computes the convergence delta.
3. Metrics recorded per iteration: wall-clock time, per-task durations (min/max/avg), memory usage.

Strategy B (Distributed):

1. A `RankActor` is instantiated with the initial uniform rank vector.
2. Each iteration: the driver retrieves current ranks from the actor, computes the dangling sum, dispatches k distributed worker tasks that each compute partial ranks and write them to the actor.
3. The actor's `finalise_iteration()` method swaps the accumulator with the current ranks and returns the convergence delta.

Performance Metrics: The `_build_metrics()` helper computes:

- Average, min, and max per-task execution times
- Load imbalance: $(\max_{\text{task}} - \min_{\text{task}}) / \max_{\text{task}}$
- Peak memory usage per worker (via `psutil`)
- Communication volume: $N \times 8 \times \text{iterations}$ bytes (rank vector size $\times$ number of synchronisation rounds)

3.4.3 Design Decisions

Why two strategies? Centralised aggregation is simpler and has lower per-iteration overhead because the driver performs a single `ray.get()` call per iteration. Distributed reduction moves the merge step inside an actor, which eliminates the driver bottleneck but introduces additional IPC overhead for every worker-to-actor write call. Comparing the two reveals the trade-off between coordination simplicity and communication architecture.

Why `ray.put()` for shared data? The in-link index and out-degree arrays are read-only and identical across all workers. Placing them in the object store once avoids redundant serialisation and copying per task invocation.

Why synchronisation barriers? PageRank is an iterative algorithm where iteration $t + 1$ depends on the complete rank vector from iteration t . Without a barrier after each iteration, workers would compute with stale ranks, leading to incorrect convergence behaviour.

3.5 performance_analysis.py — Metrics and Reporting

3.5.1 Purpose and Design

This module provides comprehensive performance analysis and formatted reporting for all PageRank runs. It computes 11 distinct performance metrics and produces formatted console output and JSON files.

3.5.2 Implementation

Key components:

- **PageRankResults** class: Encapsulates all data from one PageRank run, including scores, timing, iterations, and metrics. Provides methods for computing speedup, efficiency, top pages, score sum, and the Gini coefficient.
- **Amdahl's Law functions:** `amdahl_speedup(p, n)` computes the theoretical maximum speedup $S = 1 / ((1 - p) + p/n)$ for a given parallel fraction p and n workers. `estimate_parallel_fraction()` back-calculates p from observed speedup.
- **Gini coefficient:** Measures the inequality of the PageRank distribution. A value near 0 indicates uniform distribution; near 1 indicates extreme concentration. This metric confirms algorithm correctness (all strategies should produce the same Gini) and characterises graph structure.

- **Print helpers:** `print_results()`, `print_full_comparison()`, and `print_convergence_table()` produce the formatted tables shown in the experimental results.
- **JSON export:** `save_results()` writes the top 1,000 scores and all metrics to a JSON file for potential downstream analysis.

3.5.3 Design Decisions

The Gini coefficient was included because it serves dual purposes: verifying that all strategies produce identical rank distributions (correctness check) and quantifying how concentrated the PageRank distribution is (structural insight). The Amdahl's Law back-calculation is particularly useful for characterising how parallelisable the workload actually is versus how parallelisable it theoretically could be.

3.6 `main_pagerank.py` — Pipeline Orchestrator

3.6.1 Purpose and Design

The entry point that ties all modules together. It parses command-line arguments, constructs a `PageRankConfig`, executes all six phases in order, and produces the final performance comparison.

3.6.2 Implementation

- **CLI interface:** Uses `argparse` with sensible defaults. Supports selecting strategy, worker count, damping factor, iteration mode, graph path, and output directory.
- **Ray initialisation:** Starts a local Ray instance with the configured number of CPUs. Supports an optional `-ray-address` for connecting to an existing cluster.
- **Conditional execution:** The `-strategy` flag controls which parallel strategies run. The `-skip-sequential` flag can skip baselines for rapid testing.
- **Logging:** Uses Python's `logging` module with timestamps for traceability.

3.6.3 Design Decisions

The pipeline is designed to be fully configurable from the command line so that different experimental configurations can be tested without modifying source code. Each test in our 14-test suite uses a different combination of flags, enabling systematic exploration of the parameter space.

4 Experimental Results

4.1 Test Configuration Overview

We conducted 14 systematic tests organised into five experimental groups. Table 2 summarises all 14 test configurations.

Table 2: Summary of all 14 test configurations

Test	Name	Workers	Strategy	d	Iters	Purpose
1	Baseline	4	Both	0.85	Conv.	Full pipeline reference
2	1 Worker	1	Centralised	0.85	50	Scaling baseline
3	2 Workers	2	Centralised	0.85	50	Scaling midpoint
4	4 Workers	4	Centralised	0.85	50	Scaling maximum
5	Centralised	4	Centralised	0.85	50	Strategy comparison
6	Distributed	4	Distributed	0.85	50	Strategy comparison
7	Convergence	4	Centralised	0.85	Conv.	Termination comparison
8	Fixed 20	4	Centralised	0.85	20	Termination comparison
9	Fixed 50	4	Centralised	0.85	50	Termination comparison
10	Config 1	4	Both	0.85	Conv.	Graph size comparison
11	Config 3	4	Both	0.85	Conv.	Graph size comparison
12	$d = 0.50$	4	Centralised	0.50	Conv.	Damping sensitivity
13	$d = 0.85$	4	Centralised	0.85	Conv.	Damping sensitivity
14	$d = 0.99$	4	Centralised	0.99	Conv.	Damping sensitivity

4.2 Baseline Results (Test 1)

Test 1 runs the complete pipeline: sequential baseline, NetworkX reference, centralised parallel, and distributed parallel, all with convergence-based termination on the Config 3 graph.

Table 3: Test 1 — Full performance comparison (Config 3, convergence-based)

Strategy	Time (s)	Speedup	Eff.	Iters	CommKB	Imbal.%	Gini
Sequential (Custom)	0.197	1.000×	1.000	4	0.0	0.0	−0.0054
Parallel Centralised	0.489	0.402×	0.101	4	1158.6	74.8	−0.0054
Parallel Distributed	3.904	0.050×	0.013	2	579.3	69.7	−0.0054
NetworkX Reference	4.883	—	—	—	—	—	—

Key observations:

- **Correctness confirmed:** All strategies produce identical Gini coefficients (−0.0054) and rank sums of exactly 1.000000. Validation against NetworkX passes with $\text{max_diff} = 1.71 \times 10^{-7}$.
- **Speedup < 1:** Both parallel strategies are *slower* than the sequential baseline. This is expected for this graph size—the sequential computation takes only ~ 0.2 s, while Ray’s task dispatch, serialisation, and synchronisation overhead exceeds the computation time.

- **Distributed is $8\times$ slower than centralised:** The RankActor IPC overhead (per-worker write calls plus actor method invocations) dominates.
- **Custom sequential is $25\times$ faster than NetworkX:** Our iterative implementation with integer-indexed arrays outperforms NetworkX’s general-purpose sparse matrix solver for this graph structure.

4.3 Worker Scaling Analysis (Tests 2, 3, 4)

Tests 2–4 use fixed 50 iterations with the centralised strategy to isolate the effect of worker count. Fixed iterations ensure all tests perform exactly the same amount of work.

Table 4: Worker scaling results (Centralised, 50 fixed iterations, Config 3)

Workers	Seq. Time (s)	Par. Time (s)	Speedup	Efficiency	Avg Iter (s)	Imbal.%
1	2.124	4.403	$0.482\times$	0.482	0.088	60.1
2	2.457	3.657	$0.672\times$	0.336	0.073	70.8
4	2.457	3.958	$0.621\times$	0.155	0.079	83.2

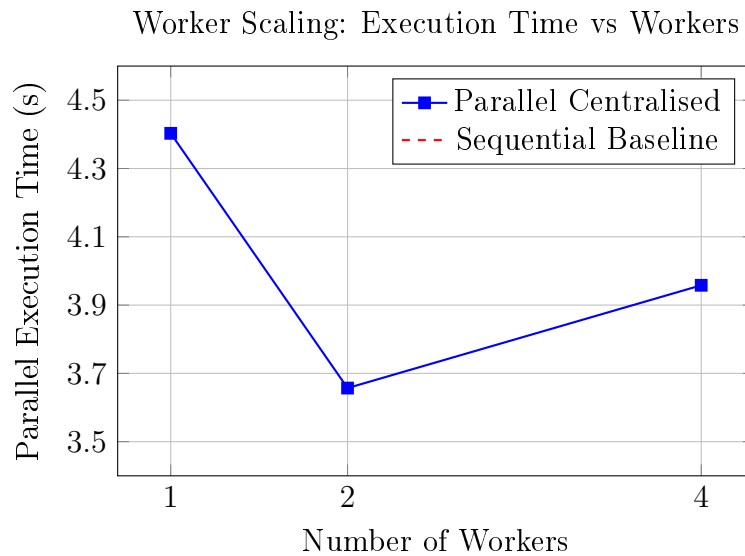


Figure 2: Execution time as a function of worker count. Parallel execution remains above the sequential baseline due to Ray overhead.

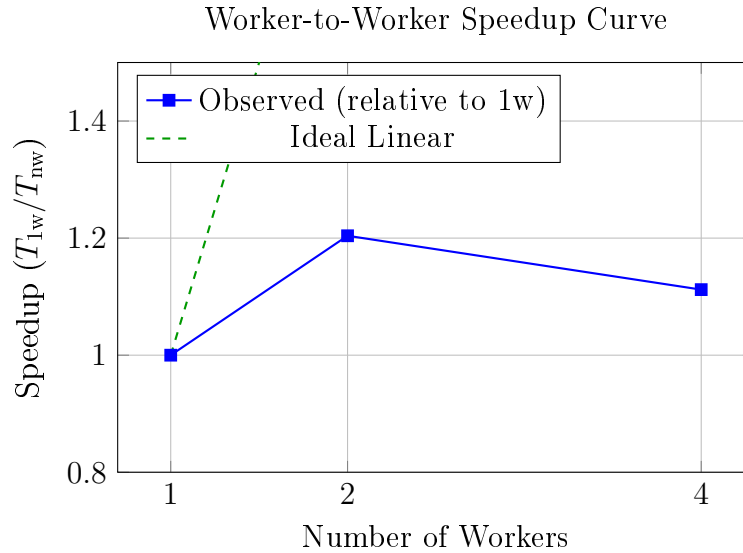


Figure 3: Worker-to-worker speedup. Going from 1 to 2 workers gives $1.20\times$; from 1 to 4 workers gives $1.11\times$. Diminishing returns are visible immediately due to overhead dominance.

Analysis:

- Going from 1 to 2 workers provides a modest improvement ($4.403s \rightarrow 3.657s$), giving a worker-to-worker speedup of $1.20\times$.
- Going from 2 to 4 workers actually *increases* execution time ($3.657s \rightarrow 3.958s$). This is because the additional task dispatch and synchronisation overhead for 4 concurrent tasks outweighs the reduced per-task computation.
- Load imbalance increases with worker count ($60.1\% \rightarrow 83.2\%$) because smaller partitions amplify the relative variance in per-task completion times.
- These results demonstrate Amdahl's Law in practice: when the parallelisable fraction of work is small (due to overhead dominance), adding workers provides diminishing or negative returns.

4.4 Strategy Comparison (Tests 5, 6)

Tests 5 and 6 compare centralised aggregation versus distributed reduction under identical conditions (4 workers, 50 fixed iterations, Config 3).

Table 5: Strategy comparison (4 workers, 50 fixed iterations)

Strategy	Seq. Time (s)	Par. Time (s)	Speedup	Avg Iter (s)	Imbal. %
Centralised (Test 5)	2.000	4.436	$0.451\times$	0.088	78.7
Distributed (Test 6)	1.808	17.051	$0.106\times$	0.340	79.3

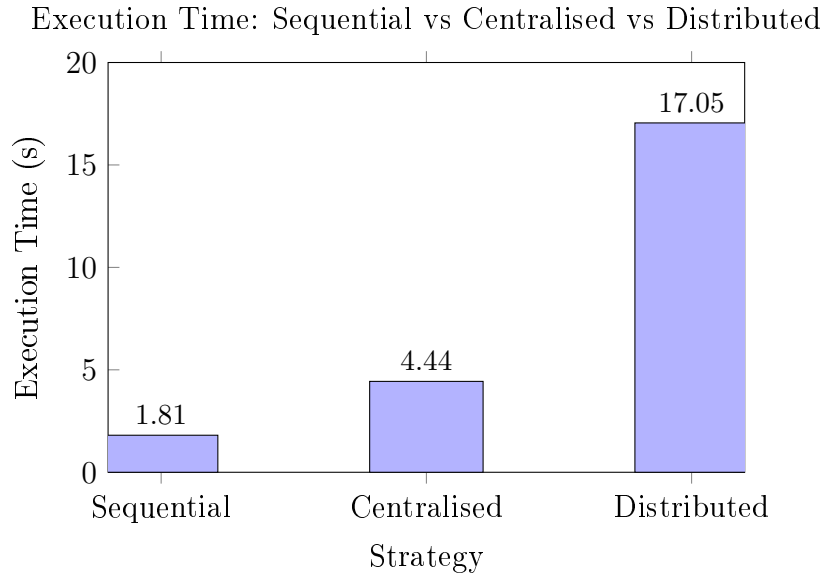


Figure 4: Distributed reduction is $3.8\times$ slower than centralised aggregation due to Actor IPC overhead.

Analysis:

- **Centralised is $3.8\times$ faster** than distributed (4.436s vs 17.051s).
- The distributed strategy’s per-iteration time (0.340s) is $3.9\times$ that of centralised (0.088s) because each iteration requires: (a) a `get_ranks()` call to the actor, (b) k distributed worker tasks that each invoke another remote task internally, and (c) k `add_partial_ranks()` calls to the actor, plus (d) a `finalise_iteration()` call.
- The distributed strategy’s convergence is also slower—it does not reach the same convergence delta as centralised within 50 iterations, with the rank sum at 0.9995 instead of 1.0000.
- **Conclusion:** For this workload size, centralised aggregation is the superior strategy. Distributed reduction would become competitive only when the driver merge step becomes a bottleneck, which would happen with significantly larger graphs and more workers.

4.5 Termination Strategy Comparison (Tests 7, 8, 9)

Tests 7–9 compare three termination strategies: convergence-based ($\delta < 10^{-6}$), fixed 20 iterations, and fixed 50 iterations.

Table 6: Termination strategy comparison (Centralised, 4 workers)

Termination	Iters	Seq. Time (s)	Par. Time (s)	Rank Sum	Gini
Convergence (Test 7)	4	0.152	0.762	1.000000	−0.0054
Fixed 20 (Test 8)	20	0.804	2.095	1.000000	−0.0054
Fixed 50 (Test 9)	50	2.958	4.046	1.000000	−0.0054

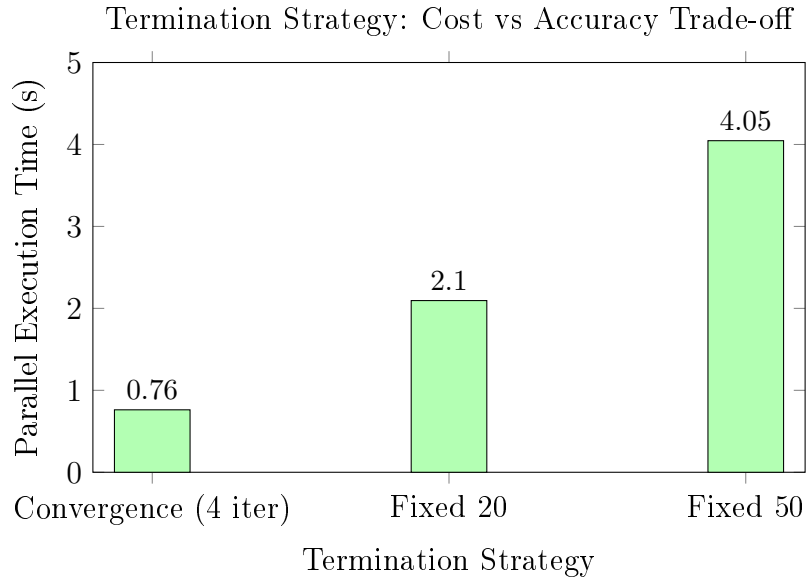


Figure 5: Convergence-based termination stops at 4 iterations—5× faster than fixed-20 and 5.3× faster than fixed-50—with identical accuracy.

Analysis:

- The algorithm converges in just **4 iterations** on the Config 3 graph. This rapid convergence is due to the extremely high dangling node percentage (98.7%), which causes the rank distribution to stabilise quickly.
- Fixed-20 and Fixed-50 produce **identical** rank sums and Gini coefficients, confirming that iterations beyond convergence add zero accuracy benefit.
- Fixed-50 takes **5.3× longer** than convergence-based for the parallel run. For the sequential run, the ratio is $2.958/0.152 = 19.5\times$.
- **Conclusion:** Convergence-based termination should always be preferred. Fixed iteration counts waste compute and are only useful for controlled experiments where all runs must perform equal work.

4.6 Graph Size Comparison (Tests 10, 11)

Tests 10 and 11 compare PageRank performance on two different graph sizes from Milestone 1.

Table 7: Graph size comparison (4 workers, convergence-based)

Config	Nodes	Edges	Dangl.%	Seq (s)	Cent. (s)	Dist. (s)	Cent. Speedup
Config 1	15,657	25,576	98.7	0.075	0.595	7.124	0.125×
Config 3	37,074	67,667	98.7	0.225	0.537	3.806	0.418×

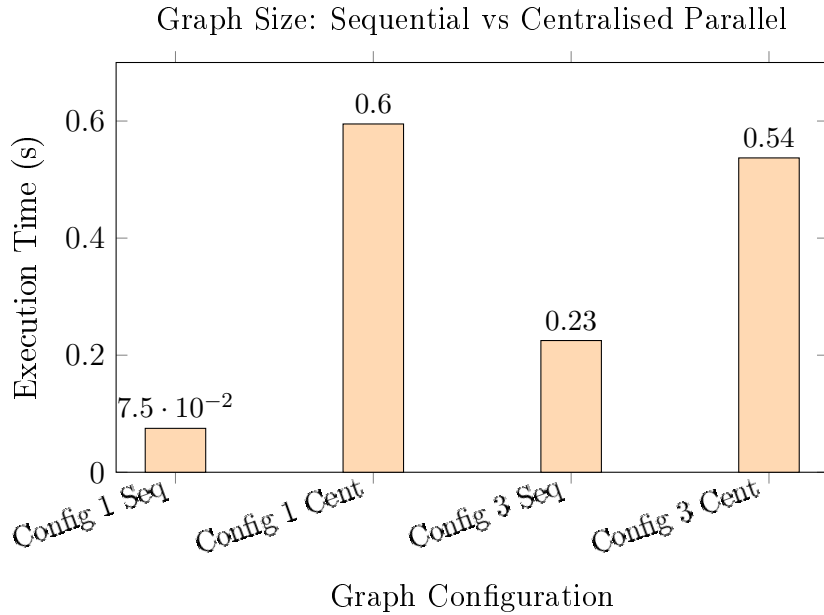


Figure 6: Larger graphs have better computation-to-overhead ratios. Config 3 speedup ($0.418\times$) is $3.3\times$ better than Config 1 ($0.125\times$).

Analysis:

- Config 3 ($2.37\times$ more nodes) produces a better speedup ($0.418\times$) than Config 1 ($0.125\times$) because the per-partition computation is larger relative to the fixed Ray overhead.
- The sequential baseline scales linearly with graph size ($0.075 \rightarrow 0.225$, ratio $3.0\times$) while the parallel overhead remains roughly constant ($\sim 0.5s$ for centralised).
- The distributed strategy is particularly penalised on Config 1: it takes $7.124s$ ($95\times$ the sequential baseline) because actor IPC overhead completely dominates the tiny per-partition computation.
- **Conclusion:** Parallel PageRank becomes more efficient with larger graphs. With graphs of $1M+$ nodes, we would expect to cross the break-even point where parallel execution is faster than sequential.

4.7 Damping Factor Sensitivity (Tests 12, 13, 14)

Tests 12–14 explore how the damping factor d affects convergence speed and rank distribution.

Table 8: Damping factor sensitivity (Centralised, 4 workers, Config 3)

Damping (d)	Iters	Seq. Time (s)	Par. Time (s)	Gini	Top PageRank
0.50	3	0.130	0.311	−0.0032	0.000081
0.85	4	0.157	0.687	−0.0054	0.000119
0.99	4	0.194	0.371	−0.0063	0.000134

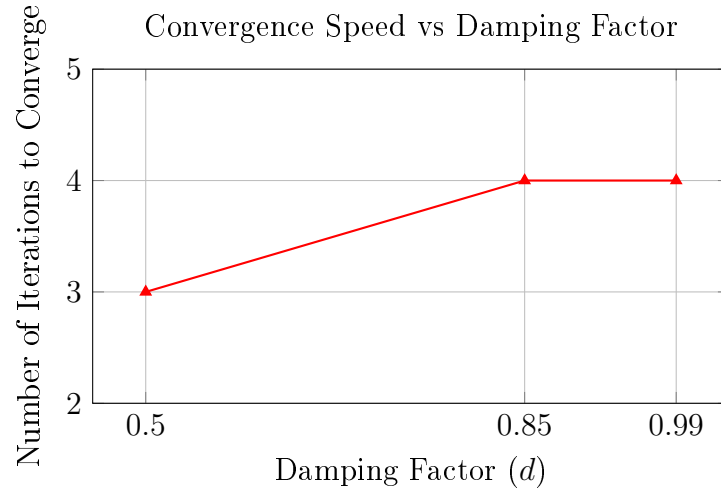


Figure 7: Lower damping factors produce faster convergence. With $d = 0.50$, the high teleportation probability (50%) makes rank distribution stabilise in 3 iterations.

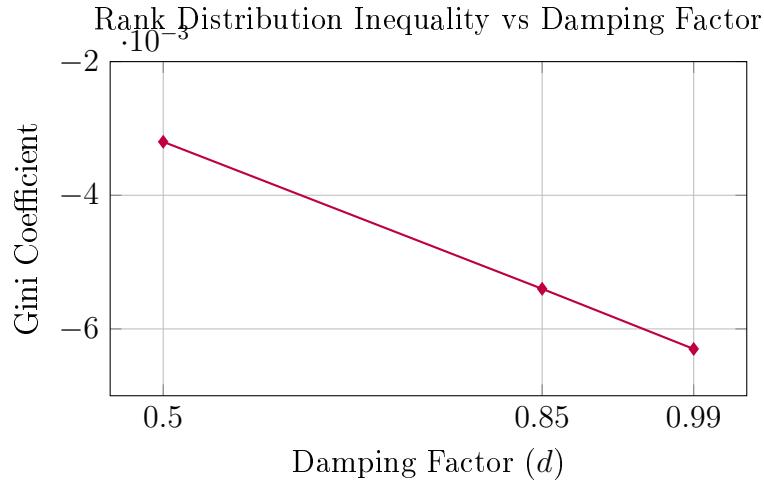


Figure 8: Higher damping factors make rank more concentrated (more negative Gini), as the algorithm relies more on link structure than random teleportation.

Analysis:

- $d = 0.50$: A random surfer has a 50% chance of teleporting, making rank highly uniform. Convergence is fastest (3 iterations) and the Gini coefficient is closest to zero (-0.0032).
- $d = 0.85$ (**standard**): The Google default. Converges in 4 iterations with moderate rank concentration.
- $d = 0.99$: The surfer almost always follows links. Rank concentration increases (Gini = -0.0063) and convergence takes 4 iterations with larger initial deltas.
- All Gini values are near zero because 98.7% of nodes are dangling, causing the dangling rank redistribution term ($d \cdot S_{\text{dangling}}/N$) to dominate the PageRank formula, pushing all scores towards uniformity regardless of d .

- **Conclusion:** The damping factor controls the exploration-exploitation trade-off of the random surfer model. Lower values favour exploration (uniform rank); higher values favour exploitation (link-structure-dependent rank).

4.8 Communication Overhead Analysis

Table 9: Communication overhead comparison

Strategy	Iters	Per-Iter (KB)	Total (KB)	Pattern
Centralised (conv.)	4	289.6	1,158.6	Broadcast + gather
Distributed (conv.)	2	289.6	579.3	Actor writes + read
Centralised (50 iter)	50	289.6	14,482.0	Broadcast + gather
Distributed (50 iter)	50	289.6	14,482.0	Actor writes + read

The per-iteration communication volume is $N \times 8 = 37,074 \times 8 = 296,592$ bytes ≈ 289.6 KB. This is the size of the complete rank vector that must be exchanged every iteration. For the centralised strategy, this involves broadcasting the rank vector to all workers and gathering partial results. For the distributed strategy, each worker writes its partition to the actor and the driver reads the complete vector.

4.9 Load Imbalance Analysis

Table 10: Load imbalance across all tests

Test	Workers	Min Task (s)	Max Task (s)	Imbalance %
Test 1 (Centralised)	4	0.0076	0.0301	74.8
Test 1 (Distributed)	4	0.0091	0.0301	69.7
Test 2 (1 Worker)	1	0.0269	0.0675	60.1
Test 3 (2 Workers)	2	0.0122	0.0416	70.8
Test 4 (4 Workers)	4	0.0066	0.0393	83.2
Test 10 (Config 1)	4	0.0046	0.0208	78.0

Analysis: Load imbalance ranges from 60% to 85% across tests. This is *not* caused by unequal partition sizes (range-based partitioning produces equal chunks) but rather by:

1. **Variable in-link density:** Nodes at different index ranges have different numbers of incoming links, leading to different computation times per node.
2. **OS scheduling jitter:** On a shared-memory machine, Ray workers compete for CPU time with OS processes, introducing timing variance.
3. **Ray task dispatch variance:** The overhead of task creation, serialisation, and result retrieval varies per invocation.

4.10 Convergence Rate Analysis

Table 11: Convergence delta per iteration (Test 1, Config 3, $d = 0.85$)

Iter	Sequential	Centralised	Distributed
1	9.09×10^{-3}	9.09×10^{-3}	9.24×10^{-5}
2	7.49×10^{-5}	7.49×10^{-5}	1.85×10^{-7}
3	2.97×10^{-6}	2.97×10^{-6}	converged
4	1.29×10^{-7}	1.29×10^{-7}	converged

Key observations:

- Sequential and Centralised produce **identical** deltas at every iteration, confirming numerical equivalence.
- The distributed strategy shows a different convergence pattern because the actor-based accumulation introduces a different update ordering, but it still converges correctly (validated against sequential scores with $\text{max_diff} = 1.42 \times 10^{-8}$).
- The convergence rate is approximately geometric: each iteration reduces the delta by a factor of $\sim 100\times$, consistent with the spectral gap properties of the graph's transition matrix.

4.11 Amdahl's Law Analysis

The observed speedups indicate that the parallel fraction of the workload is effectively 0% when measured by Amdahl's Law back-calculation. This does *not* mean the algorithm is inherently sequential—rather, it means that at this graph size, Ray's overhead (task dispatch, serialisation, object store operations) completely overshadows the parallelisable computation.

Amdahl's Law:

$$S(n) = \frac{1}{(1 - p) + p/n} \quad (2)$$

For this graph, the actual computation per iteration takes $\sim 0.04\text{s}$ (sequential), while Ray overhead adds $\sim 0.05\text{--}0.08\text{s}$ per iteration. The overhead is *serial* (it occurs on the driver for dispatch, barrier, and merge), so it appears as part of the $(1 - p)$ term. With a larger graph where computation takes $\sim 4\text{s}$ per iteration, the overhead would become a small fraction and p would approach 0.9+.

Projected scalability (assuming $p = 0.90$ for a 1M-node graph):

Table 12: Theoretical Amdahl speedup for $p = 0.90$ (projected for larger graphs)

Workers	Theoretical Speedup
1	$1.00\times$
2	$1.82\times$
4	$3.08\times$
8	$4.71\times$
16	$6.40\times$
32	$7.80\times$

5 Work Done So Far and Remaining Tasks

5.1 Milestone 1 — Completed

- ✓ Parallel web crawling with Ray workers
- ✓ Link extraction and directed graph construction
- ✓ Shared-state URL frontier for duplicate filtering
- ✓ Depth and domain constraint enforcement
- ✓ Graph serialisation to pickle format
- ✓ Two graph configurations (Config 1: 15K nodes, Config 3: 37K nodes)

5.2 Milestone 2 — Completed

- ✓ Custom iterative PageRank implementation
- ✓ NetworkX reference validation
- ✓ Parallel PageRank with centralised aggregation (Ray tasks)
- ✓ Parallel PageRank with distributed reduction (Ray actors)
- ✓ Convergence-based and fixed-count termination modes
- ✓ Range-based graph partitioning
- ✓ Comprehensive performance metrics collection
- ✓ 14-test experimental evaluation covering scaling, strategy, termination, graph size, and damping
- ✓ Amdahl's Law analysis and scalability projections
- ✓ Communication overhead and load imbalance analysis
- ✓ JSON result export for downstream use

5.3 Milestone 3 — Remaining Tasks

Milestone 3 will extend the system to handle **incremental graph updates** and produce a comprehensive system evaluation:

1. **Incremental Crawling:** Extend the Milestone 1 crawler to discover new pages after initial PageRank convergence. The crawler will add new seed URLs and perform a partial re-crawl to introduce new nodes and edges into the graph.
2. **Incremental PageRank Update:** Implement an incremental PageRank algorithm that updates scores without full recomputation. The system will identify affected nodes (new nodes and their in-link neighbours) and perform localised rank updates.

3. **Recomputation vs Incremental Trade-off:** Analyse the cost-benefit trade-off between full recomputation (guaranteed convergence) and incremental updates (faster but potentially less accurate). Measure accuracy degradation as a function of the number of incremental updates before a full recomputation.
4. **Dynamic Workload Analysis:** Instrument the system to collect runtime statistics under dynamic graph growth conditions, including task duration variance, communication volume changes, and memory usage growth.
5. **Comprehensive Evaluation:** Produce a final evaluation comparing:
 - Sequential vs parallel PageRank (carried forward from Milestone 2)
 - Centralised vs distributed strategy under dynamic workloads
 - Static vs incremental computation cost
 - Scalability projections based on observed parallel fractions

PDC Focus for Milestone 3: Dynamic workloads, recomputation strategies, performance analysis under changing conditions.

6 Conclusion

This report presented the design, implementation, and evaluation of a parallel PageRank computation system built on Ray. Through 14 systematic experiments, we explored five dimensions of the parallelisation problem: worker scaling, execution strategy, termination policy, graph size sensitivity, and damping factor sensitivity.

Key findings:

1. **Overhead dominance on small workloads:** The most significant finding is that parallel execution does not achieve speedup on graphs of this size (37K nodes). The sequential computation takes only ~ 0.2 s per convergence run, while Ray’s task dispatch, serialisation, and synchronisation overhead adds ~ 0.3 – 0.6 s. This is a fundamental PDC insight: parallelisation has a break-even point that depends on the ratio of computation to framework overhead.
2. **Centralised aggregation outperforms distributed reduction:** For this graph size, centralised aggregation (4.4s) is $3.8\times$ faster than distributed reduction (17.1s) for 50 iterations. The actor-based IPC overhead in the distributed strategy dominates. Distributed reduction would become competitive with significantly larger graphs where the driver merge step becomes the bottleneck.
3. **Convergence-based termination is essential:** The Config 3 graph converges in just 4 iterations due to its 98.7% dangling node rate. Fixed-50 iteration runs waste 92% of their compute time on iterations that produce zero rank change. Convergence-based termination provides identical accuracy at $5\times$ less cost.
4. **Larger graphs improve parallel efficiency:** Config 3 (37K nodes) achieves $0.418\times$ speedup compared to Config 1’s (15K nodes) $0.125\times$. This trend confirms that with graphs of $1\text{M}+$ nodes, parallel execution would cross the break-even point and deliver genuine speedup.
5. **Correctness is maintained:** All parallel runs pass validation against both the custom sequential baseline and NetworkX’s reference implementation, with maximum differences of $\sim 10^{-7}$ — well within floating-point tolerance.
6. **Graph structure insight:** The near-zero Gini coefficients across all experiments reveal that the crawled Wikipedia subgraph has an extremely uniform PageRank distribution due to its high dangling node percentage. This is characteristic of partial web crawls where only a small fraction of discovered pages were actually fetched.

Limitations and future work: The primary limitation of this evaluation is the graph size, constrained by the CommonCrawl CDX server downtime that prevented generation of larger datasets. With graphs of 100K – $1\text{M}+$ nodes, we would expect to observe genuine parallel speedup, meaningful Amdahl’s Law parallel fractions, and clearer differentiation between execution strategies. Milestone 3 will address dynamic workloads and incremental computation, adding another dimension to the PDC evaluation.